

AgentFlame: 面向 AI 编程 Agent 系统效应的语义归因剖析

AgentSight 研究原型

2026 年 6 月 18 日

摘要

AI 编程 agent 的可观测性问题不只是“模型调用了哪个 tool”，而是用户语义意图如何导致真实系统副作用：进程、文件、网络、测试、失败重试和 token 消耗。现有 LLM observability 工具擅长展示单次运行的 trace tree、span duration、tool call 和 cost；传统 profiler 和进程工具擅长聚合系统行为。二者分开时，一个用户常问的问题仍然难答：哪个任务语义造成了哪些重复、沉重或越界的系统足迹？本文提出 AgentFlame，一种语义系统效应剖析模型：本地小模型只给 session、prompt 和 LLM call 生成一个词标签；当前 full run 从 agent-native tool logs 继承这些标签，目标模式则让 tool、shell、child process 与 file/network effect 通过确定性 lineage 继承这些标签；最终以 folded stack 形式聚合为 flamegraph。在用于本文图表的 205 个可读 Codex/Claude 真实 session headline run 上，Agent-Flame 标注 2,463 个 prompt 和 90,930 个 LLM call，0 个最终标签失败；它把 167,005 个系统观测折叠成 24,295 条语义系统栈。随后 R170 current refresh 覆盖当前可发现的 325 个本仓库 session，并记录 183,714 个 system observations、26,829 条 semantic system stacks 和 35,136 次真实 llama.cpp tag calls。去掉 session/prompt 语义后，nonsemantic baseline 有 90.219% 的观测权重落入混合多个语义区域的 buckets；flat effect baseline 的混合权重为 90.770%。R131 消融进一步显示：在保持同一 167,005 个系统观测总权重不变时，session-only 仍混合 84.180% 的完整语义权重，prompt-only 降到 37.687%，完整 session+prompt 降到 0.000%。这些结果支持一个机制性结论：语义帧，尤其是 prompt 标签，能分开传统 trace 或 process summary 会合并的 task-effect 区域。本文也明确边界：当前 full run 仍基于 agent-native session 历史；R114 在 20 个固定 codex command-mode 任务上把 1,273/1,273 个 in-scope effects 连到 agent process family，precision/recall 均为 100.0%，并拒绝 3,170 个 negative-control effects；R182 只证明 record-mode --trace-net 能 join 35/35 个低层 codex network rows；R191 进一步在固定 Codex HTTP probe 上观察到 4/4 个 target python3 bind/listen/connect rows 全部 join，且 0/310 negative-control effects 被误归因。因此用户任务实验、跨仓库规模、broader network workloads 和 full-history exact integration 仍未完成，不能声称用户效用或完整系统 provenance 已被证明。

1 问题

一个 AI 编程 agent 运行结束后，开发者通常不想逐条回放消息。他们想知道：这次运行哪里最重，哪里绕路，哪里反复试错，哪些文件或网络目标被碰过，哪类用户请求导致最多测试、最多读取、最多失败或最多 token。传统工具各自只覆盖一半。LLM trace 工具能展示 agent、LLM call、tool call、prompt、completion、latency 和 cost。进程或文件审计工具能展示 git、cargo、rg、sed、docker 等系统行为。Span-duration flamegraph 甚至已经出现在 multi-agent

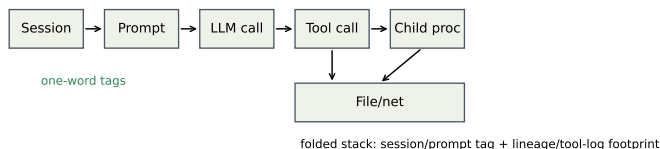


图 1: AgentFlame 的归因模型。小模型只标注 session、prompt、LLM-call 语义；tool/process/file/network effects 通过确定性 lineage 继承语义帧并折叠成 stacks。

workflow debugging 中，用于看 critical path、parallelism、error cascade 和 tool overhead。

但是这些视图仍难回答跨层问题：

为了哪个用户任务，agent 反复运行了哪些命令、读取了哪些路径、产生了哪些网络或文件副作用？

本文的核心判断是：agent observability 的对象不应只是 span，也不应只是进程，而应是 *task-grounded system effect*。我们把目标栈写成：

```
sessionTag;promptTag;llmcall/tool;process*;effect
```

这个模型把两类传统工具分开的信息合并起来。上层的 session、prompt、LLM call 用本地小模型压缩成一个词；下层 tool、process、file/network effect 不由模型猜测，而通过确定性关联继承语义上下文。最后用 folded stack 聚合重复路径，让宽度表示 event count、effect weight 或 token weight，而不是只表示 span duration。

本文的贡献不应表述为“agent flamegraph”。已有系统已经把 agent spans 画成 flamegraph。本文的贡献是一个更具体的系统归因模型：在 agent-native local histories 中把用户语义意图与系统行为代理接起来，并为 live AgentSight traces 定义 exact provenance 目标，使跨 session 的重复副作用可聚合、可检查、可比较。

2 设计

图 1 展示了 AgentFlame 的设计。语义层非常小：每个 session、prompt 和 LLM call 只生成一个 lowercase word，例如 refactor、review、test、research。这些词不是固定 ontology，也不是 verdict；它们只是给折叠栈提供短、稳定、可搜索的任务帧。

系统层不交给 LLM 判断。Agent-native 历史模式从 Codex/Claude JSONL 中恢复 tool 类型、命令入口、状态、路径组和粗 effect 类别。AgentSight exact 模式的目标输入是：

```
tool_call -> shell -> child process ->
file/network effect
```

每个 in-scope effect 必须能追溯到 tool call 和 prompt ancestry。如果只能用 PID 或时间戳模糊匹配，就不能作为强 provenance claim。因此，语义由上层继承，系统事实由 instrumentation 或 session parser 产生。

2.1 为什么是一个词

一个词标签有三个工程优点。第一，火焰图帧必须短；长摘要会让图不可读。第二，标签只用于导航和聚合，不承担安全、正确性或因果判决。第三，一个词标签容易本地化和缓存化。当前实现对 session/prompt/LLM 文本各处理一次，后续数十万条系统事件只做普通程序关联和聚合。

这也约束了论文 claim。AgentFlame 不证明标签是真实意图 ontology。它先证明一个更可审计的机制：加入这些短语义帧后，传统 baseline 会合并掉的系统效应是否被分开。

2.2 Stack 语法

系统效应栈形如：

```
project;agent;session;prompt;call:tool/...;process*;effect/path/domain;token
```

Token 栈形如：

```
project;agent;session;prompt;call:llm/...;model;kind
```

完整视图保留 session、prompt、LLM-call 语义。消融视图删除部分语义轴：session-system、prompt-system、llm-token 等。所有视图共享同一事实表；投影不应改变总权重。

2.3 可视化模型

AgentFlame 不把所有信息塞进一张图。面向开发者的核心界面应有四类互补视图。第一，semantic system flamegraph 用宽度表达 system-effect count，用于回答“哪里重、哪里绕、哪里重复”。它适合跨 session 聚合，因为 folded stack 会把同一语义任务下重复出现的 rg、cargo test、文件读取或网络目标折叠到同一横条。Duration 是单独的 projection：当前 R225 只支持 prompt wall-clock baseline，不支持 active runtime 或 tool/LLM span-duration。第二，token flamegraph 使用同样的 session/prompt/LLM-call 语义帧，但叶子变成 model/prompt/completion token，回答“哪类任务消耗上下文和模型预算”。

第三，exact-lineage tree 不是聚合图，而是 provenance drill-down。它展示一个 prompt 下的 tool call、shell、child process 和 file/network effect 链，用来确认某个横条是否真的来自目标 agent process family。R182/R183 说明这个视图不能省略：低层 codex network rows 可以被 join，但只有 R191 这种 target python3 child-process rows 也被观察并 join 后，UI 才能把对应固定 workload 标成 covered。第四，claim/evidence board 把每个 claim 连接到 artifact、oracle 和 status，防止用户把 smoke run 当成完整结论。

因此 flamegraph 与 tree 的职责不同。Flamegraph 负责发现热点和重复模式；tree 负责解释一个热点能不能被信任；claim board 负责告诉用户哪些结论还只是 partial。这种三层设计是传统 profiler 和普通 agent trace 都缺的：前者没有 prompt 语义，后者通常没有系统副作用的 deterministic ancestry，更不会把 claim boundary 作为一等视图。

3 实现

本文的实验结果来自 AgentFlame 研究原型及其生成 artifacts。该原型当时是独立 Rust CLI，并输出 agentflame.json、tags.json、folded stack 文件、SVG 和 HTML dashboard。当前开源仓库的产品化入口已经收敛到 agentpprof/：它复用 agent-session，输出 pprof-compatible .pb.gz、folded、SVG 和 JSON。因此，本文的

headline 数字评估的是 AgentFlame 研究原型；agentpprof 是产品化方向，除 smoke artifacts 外还没有替代本文所有 RQ1-RQ6 结果。默认输出路径是：

```
.agentsight/agentflame/latest/
```

本次 full run 的命令为：

```
cargo run --manifest-path
agentflame/Cargo.toml -- run --project-root
. --scan-files 10000 --max-sessions
10000 --llama-url http://127.0.0.1:18080
--model local --timeout 60 --out
.agentsight/agentflame/latest
```

这条命令是研究 artifact 的 provenance，不是当前仓库的入口。当前仓库入口示例是：

```
cargo run --manifest-path
agentpprof/Cargo.toml -- --project-root .
-o agent.pb.gz
```

模型为本地 qwen2.5-3b-instruct-q4_k_m.gguf，通过 llama.cpp server 运行，temperature 为 0，并用 grammar 约束输出一个词。AgentFlame 没有 heuristic fallback：如果 LLM server 缺失，或模型重试后仍不能输出合法一词标签，run 失败。

3.1 隐私与可复现

原始 agent trace 不提交。agentflame.json 默认包含 redacted previews、hash、tag、计数和路径组；tags.json 保存 tag cache 和 LLM provenance，不保存原始 prompt 文本。本次 run 遇到一个 root-owned Claude JSONL，不可读；系统没有要求 root 权限读取，而是跳过并写入 warnings。这让覆盖范围可审计，而不是静默假装完整。

3.2 与 AgentSight 的关系

AgentFlame 的第一模式是零 instrumentation 的历史分析。AgentSight 的独特点是运行时 exact provenance：tool_call -> shell -> child process -> file/network effect。正确集成后，AgentFlame 负责语义帧，AgentSight 负责系统效应事实，两者通过 tool/session/prompt ancestry 连接。当前 full run 还不是 live exact-effect 结果。R110-R113 将最小 session/tool envelope 从 fixture、native export、DB backfill 推到 record -- <command> 的 capture path；R114 用 20 个真实 Codex command-mode 任务验证该路径，加入 per-task negative controls，并在 scoped oracle 下达到 1,273/1,273 in-scope effects joined、0 false positives、0 false negatives。R182 暴露并修复 record-mode process runner 没有传 --trace-net 的缺口；修复后 35/35 个低层 codex process network rows 全部 join、0/604 negative controls 被 join，但 target-specific loopback/expected child-process network rows 为 0/0。R191 随后修复两个更细的 capture 边界：record envelope start time 必须使用 target process start timestamp，process filter 必须允许 session 内进程或 tracked PID descendants；在固定 Codex HTTP probe 上，4/4 个 target python3 network rows 全部 join，0/310 negative controls 被 join，precision/recall 为 100.0%/100.0%。C4 因此支持固定 command-mode suite 和一个固定 target-network workload；它仍不是 full-history、跨仓库、broader network workload 或 HTTP payload/URL reconstruction 证据。

表 1: 用于论文图表的本机 AgentSight headline session 运行指标。R170 另行记录当前 325-session refresh。

指标	数值
Readable sessions	205
Codex / Claude / Claude-subagent	78 / 50 / 77
Raw tool events	130,632
Raw LLM events	90,930
Prompt rows	2,463
Unique prompt tags	303
LLM-call tags	90,930
Unique LLM-call tags	1,250
Tag requests	93,598
Cache hits	64,297
llama.cpp HTTP calls	29,302
Successful final uncached tags	29,301
Final tag failures	0

4 评估

4.1 RQ1: 本地小模型能否全量标注？

表 1 给出用于本文图表的 headline full run 结果。Agent-Flame 分析了 205 个可读 session, 其中 Codex 78 个、Claude 50 个、Claude subagent 77 个。它处理 2,463 个 prompt rows 和 90,930 个 LLM-call events。标签器共有 93,598 个请求, 64,297 个命中 cache, 29,302 次真实 llama.cpp HTTP calls, 29,301 个成功最终 uncached tags, 0 个最终失败。HTTP calls 比成功最终 tags 多 1 次, 是一次被 retry 吸收的中间无效输出。Prompt 和 LLM-call tag 的非法计数均为 0。这说明一词语法和本地 3B 模型路径在真实历史规模上可运行。R170 current refresh 在不覆盖 latest 图表输入的前提下重新扫描当前本机历史, 覆盖 325 个 session、142,468 个 raw tool events、114,837 个 raw LLM events、183,714 个 system observations、26,829 条 semantic system stacks, 并产生 35,136 次 fresh llama.cpp calls 且 0 个 tagger failures。因此, 205-session run 是论文图表基准, R170 是当前全量本机历史刷新证据; 二者都不是 C5/C6 outcome evidence。R180 进一步表明, 在同一组 300 个 redacted fragments 上, 本机可用的 0.6B、1.1B 和 3B GGUF 都能输出符合语法的一词标签; 但该实验只支持 syntax/latency/stability, 不支持 semantic adequacy。

这个结果不等于标签语义充分。例如某些 tag 明显带噪声: agentsightism、testcodex、bashoutput。因此系统需要区分 raw tag 和 canonical display tag。R189 在 R170 current refresh 上实现了第一版可审计合并层: raw tag 永远保留, 图和聚合默认可使用 canonical tag。它把 prompt-effect tag 从 263 个合并到 216 个, 把 prompt-row tag 从 328 个合并到 279 个, 把 LLM-event tag 从 1,423 个合并到 1,254 个; 同时保持 folded 总权重不变, system stacks 从 26,829 条降到 26,067 条, token stacks 从 8,569 条降到 7,661 条。这说明存在一个可审计的候选长尾降噪空间, 但不能证明这些 raw tags 一定语义冗余。因此 AgentFlame 使用 semantic compaction lifecycle: raw tag 不变, canonical map 只作版本化展示层, pending actions 保存 keep/merge/regenerate/split, 小模型生成的 regenerated_tag 只有经过 R203 paired/adjudicated promotion review 和 display-map diff 后才能进入 canonical view。R209 将这个 lifecycle 落成可消费数据契约: active display map 每个 raw tag 一行, drilldown index 保留 raw 支持和 top system profile, reviewed diff 只在人工 promotion 后产生。因此 RQ1 支持的是 feasibility 和 syntax claim;

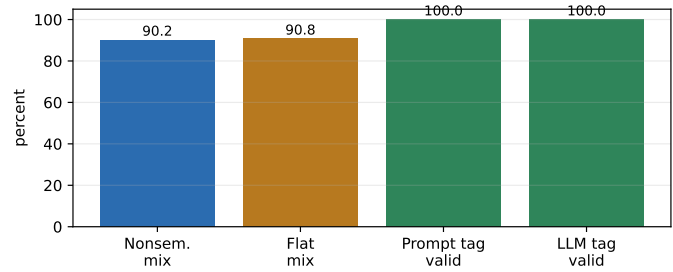


图 2: 当前机制结果。图由 docs/visexp/paper/make_figures.py 从 .agentsight/agentflame/latest/agentflame.json 重新生成; 它反映 Rust full-run 输出, 而不是旧的 Python sampled-run artifact。

表 2: 语义栈与 baseline mixing。

指标	数值
System observations	167,005
Unique semantic system stacks	24,295
Semantic compression	6.874x
Max stack reuse	6,004
Nonsemantic mixed buckets	4,209
Nonsemantic mixed weight	90.219%
Flat mixed buckets	4,051
Flat mixed weight	90.770%

tag adequacy 需要单独实验。

4.2 RQ2: projection 方法如何取舍？

Full run 产生 167,005 个 system observations, 并折叠成 24,295 条 unique semantic system stacks, 压缩率为 6.874 倍。关键不是压缩本身, 而是 baseline mixing。当删除 session/prompt 语义帧后, nonsemantic folded baseline 有 4,209 个 mixed buckets, 覆盖 150,670 个观测, 也就是 90.219% 的系统权重。如果进一步退化成 flat effect baseline, mixed weight 为 151,590, 占 90.770%。

这为回应“传统工具也能看出来”的质疑提供了机制性证据。传统 flat summary 可以告诉我们 sed read 有 25,755 次、rg read 有 15,336 次、cargo test 有 2,903 次。但是它不知道这些行为分别来自 refactor、review、research、design 还是 test prompt 区域。AgentFlame 的价值是把 heavy/repeated system effects 按任务语义拆开。

R211 将当前 R170/R189 artifact 转成可放入论文的 label distribution 和 baseline-collapse examples。在 R170 full-history 刷新中, rg 覆盖 176 个 prompt tags, sed 覆盖 180 个, git 覆盖 147 个, cargo 覆盖 68 个。具体 collapsed key process:git;effect:read;status:ok 有 116 个 prompt tags, top prompt 只占 24.977%; process:cargo;effect:test;status:ok 有 48 个 prompt tags, non-top-prompt weight 为 68.05%。这说明传统 process/effect summary 不只是少了一层标签, 而是把大量不同用户意图导致的相同行为合并成同一个 bucket。R211 同时输出 tag-distribution-r211.svg 和 process-splits-r211.svg, 用于绘制本文的 baseline failure/semantic split 图; 该 artifact 不调用 LLM, 也不读取 raw trace。

R131 把这个结论进一步做成轴级别消融。R224 用同一个 checker 在 R170 current refresh 上重跑该消融, 使 semantic-axis rows 与 R212 display-policy rows 共享 183,714 个 system-effect denominator。每个 variant 都从同一个 folded input 投影而来, 并检查总权重保持不变。同时, checker 记录 agentflame.json totals 与 folded inputs

表 3: R224/R170 system-effect 语义轴消融。所有行都保持 183,714 总权重。

Variant	Unique stacks	Bucket	Residual
No semantic	11,967	90.402%	44.716%
Session only	15,027	84.407%	33.434%
Prompt only	24,703	36.722%	7.485%
Session + prompt	26,829	0.000%	0.000%

匹配, 并验证已有 nonsemantic/session/prompt folded files 与 projection exact match。表 3 显示, system-effect 分割主要来自 prompt 轴: session-only 只把 mixed full-semantic bucket weight 从 90.402% 降到 84.407%, prompt-only 则降到 36.722%。如果只计算 mixed buckets 中非主导 full semantic variants 的 residual weight, prompt-only 也从 no-semantic 的 44.716% 降到 7.485%。完整 session+prompt 视图的 0.000% mixed weight 是定义上的上界, 不应被解释为用户效用; 它只说明 full semantic key 没有在投影中被合并。

R223 将 RQ2 从“语义栈是否更好”改写成更一般的 projection-selection 问题: 在 R170-derived generated evidence 上, stack schema、display policy 和 tag vocabulary overlay 都应是可插拔选择。该实验只读取 R224/R205/R209/R212/R219 已生成 artifacts, 不读取 raw trace, 不调用 LLM, 也不评分人工 outcome。因此, R223 的 baseline 是 count-weighted process/effect projection, 而不是真实 span-duration workflow flamegraph; 后者回答 critical path/duration 问题, 应作为 C5 用户任务实验中的独立 baseline, 而不是用当前 folded effect counts 替代。它给出的结论不是单一最佳火焰图, 而是分层默认: no-semantic 适合作为 system-hotspot baseline, 因为它只有 11,967 个 stacks、压缩率 15.352x, 但 90.402% 的 effect weight 被混合; prompt-only 是 system-effect profile 的最佳单语义轴, 因为它把 mixed/residual mixed weight 降到 36.722%/7.485%, 同时保持 7.437x 压缩; 完整 session+prompt 是 audit 和 drilldown 视图。在 display policy 上, R209 conservative display 与 alias-only 等价, 只有 26,612 个 stacks 且 0.0% unreviewed active weight; 更激进的 profile-guarded candidate view 可以降到 26,067 个 stacks, 但会激活 2.532% 未审查 effect weight, 因此只能作为上界消融, 不能做默认。在 vocabulary overlay 上, canonical display 把 raw unique tag strings 从 1,546 降到 1,364, top-20 support coverage 从 93.683% 提升到 95.186%, 同时保留 raw drilldown。这个 coverage 只是 display-support concentration, 不是 tag correctness; 在 R124/R190/R203 人工标签返回前, 不能把它写成 tag adequacy、merge quality 或 promotion quality。R225 进一步补上一个 duration 视角的边界实验: 它只读取 R170 agentflame.json 和 semantic-system.folded.txt, 从 prompt timestamps 重构 2,858 个 prompt wall-clock spans, 生成 duration-weighted folded stack 和 SVG。这个 baseline 覆盖 324/325 个有 prompt spans 的 sessions, 并且 covered prompt-index effects 覆盖 183,714/183,714 个 system-effect observations; 总 prompt wall-clock duration 为 859.019 小时; duration top-10 与 effect-count top-10 只重合 7 个, top-20 重合 12 个, prompt-tag rank Spearman correlation 为 0.623。例如 network 在 duration 中排第 8、占 2.837%, 但在 system-effect weight 中排第 93、占 0.040%; benchmark 则在 effect weight 中排第 7、占 2.195%, 但 duration 中排第 34、占 0.198%。这说明 duration flamegraph 与 system-

effect profile 相关但回答不同问题。同时, R225 明确记录历史 artifact 没有 tool/LLM start-end span, 因此它不是 active runtime、完整 workflow span-duration trace, 也不能支持 C5。这就是系统设计需要可插拔 projection 的原因: 不同视图优化不同问题, 而不是让小模型一次性决定最终语义真相。

4.3 Case Study: 能看出什么

本次 full run 中, 最大的 semantic stack 是:

```
project:agentsight;agent:codex;session:refactor;prompt:refa
```

权重为 6,004。另一个明确的系统例子是 cargo test。Flat summary 只会显示 2,903 次 successful cargo test。Nonsemantic stack 会保留 call:tool/shell;process:cargo;effect:test;status:ok, 但仍把不同语义区域混在一起。Semantic stack 将它拆成多个区域, 包括 review/review、refactor/refactor、research/explain、research/refactor、design/review 等。R211 的 R170 刷新版本进一步显示 process:cargo;effect:test;status:ok 在 48 个 prompt tags 中分布, refactor 只占 31.95%, 其余 68.05% 来自 review、research、commitlog、explain、test 等 prompt。这正是 span-duration flamegraph 或 flat process table 难以回答的问题: 同样是 test process, 语义来源不同, 调试结论也不同。

这类结果生成可供检查的候选分解, 用来辅助三类 developer questions:

- 哪个任务导致最多重复测试或读取?
- 哪些 prompt tag 反复触碰同一组路径或域名?
- 哪些失败行为集中在特定 semantic region, 而不是全局系统问题?

Trace tree 能回放某次调用, span flamegraph 能看 duration bottleneck, flat summary 能看重命令。但它们都不直接给出跨 session 的 task-effect 聚合。

4.4 RQ3: exact lineage 是否成立?

当前答案是: 固定 command-mode suite 已经成立, 但 broad full-history provenance 仍不足。effect_lineage_smoke.py 先在 AgentSight-shaped fixture 上验证 process/file/network effects 可以连到 session/tool/prompt ancestry。R110-R112 把同一思想推进到 3 个真实 AgentSight SQLite exports: 先由 harness 增加 agent-run envelope, 再移入 native export, 最后写入 DB 副本; 这些旧 snapshots 只能 join 182/318 raw effects, 说明 envelope 存在但覆盖还不够。R113 把 record -- <command> 的 agent-run envelope 放到 capture path; R113-live 随后在 5 个真实只读 codex exec 任务上验证该路径能创建 5/5 sessions/tools 并 join 508/508 raw effects。R114 将这个路径扩展到 20 个固定 Codex tasks, 包括 read-only、edit、test/debug、dependency、failure/retry 和 disposable-workspace write tasks, 并加入 per-task negative-control bursts。在 scoped oracle 下, R114 结果为: 20/20 target tasks completed, 20/20 tasks observed negative controls, 1,273/1,273 in-scope effects joined, 0 false positives, 0 false negatives, precision 和 recall 都是 100.0%, 3,170 个 observed negative-control effects 中 0 个被 join。Raw join 仍只有 22.055%, 这是预期结果: wrapper、sibling 和 out-of-scope effects 应该保持 orphan, 而不是被

错误归因给 agent。R182 进一步检查 network-effect case。第一次 loopback run 暴露出 `agentsight record` 没有给 process eBPF runner 传 `--trace-net`；修复后，35/35 个低层 `codex` process network audit rows 全部 join，network orphan 为 0，precision/recall 仍是 100.0%，0/604 negative controls 被 join。严格 oracle 同时显示 target-specific loopback/expected child-process network rows 为 0/0；因此 R182 只是 record-mode `--trace-net` 实现证据。R191 将 workload 固定为预置 Python HTTP probe，并修复 envelope timestamp 与 session-hard-filter 问题；该 run 观察到 4 个 target python3 network effects (bind, listen, connect)，4/4 全部 join，0/310 negative controls 被 join，scoped precision/recall 为 100.0%/100.0%。R191 的 broad effect-lineage smoke 仍因 wrapper/out-of-scope orphan 返回非零，因此论文只把它作为 scoped target-network oracle。它补上固定 target-network workload 的 C4 evidence，但仍不是 HTTP URL 语义恢复或任意 network workload 证明。

这个结果已经支持“固定 command-mode suite 中 exact semantic-effect lineage 成立”。它仍不能支持“任意历史、任意仓库、任意 agent 都有完整 provenance”。完整 OSDI artifact 还需要跨仓库 replication、更多 agent 类型、更丰富的 network workloads、HTTP 语义恢复、child-depth/path/domain breakdown，以及用户任务结果。

4.5 RQ4: 用户效用是否成立？

当前答案是不成立，或者更准确地说：未验证。已有 task packet、answer key、scorer 和 launch package，但没有真实参与者响应。R184 机械 gate 把 C5 状态记录为 `ready_for_participant_collection`，而不是 `supported`。R186 计划审查进一步确认：下一步应先运行真实 R142 五人 pilot，然后才进入 R151 纸面规模实验。R187 已把冻结的 R142 assignment 打包成 P01-P05 参与者文件和空白 70 行 response CSV；manifest 检查 launch 目录没有 answer key、参与者 payload 没有 oracle/scoring 字段，且 `real_response_count=0`、`c5_supported=false`。R193/R195/R207 进一步把这个收集路径变成可执行 handoff：R193 汇总空白 R124/R190/R203 label sheets 和 R142 launch 指针，R195 定义返回 CSV 的 ingestion/scoring contract，R207 检查五个 R142 participant packets、70 行空白 response template、两份 300 行 R124 sheets、两份 160 行 R190 sheets、两份 41 行 R203 sheets 以及 R195 inbox 文件名均有效。这些 artifact 只支持 launch readiness；它们仍记录 0 个参与者响应、0 个人工标签，不能支持 C5。当前 baseline 应设为 trace tree、event-count-proxy、flat process summary、nonsemantic folded stack 和 semantic folded stack，比较 accuracy、time、false positives 和 confidence。当前 R142 packet 中原来的 span-like 条件已经明确命名为 event-count proxy，因为 folded artifact 没有真实 duration；它不能直接当作 span-duration baseline。R225 已经从 R170 timestamps 重构成 prompt wall-clock duration baseline，并显示它与 effect-count profile 的 top tags 和排序有明显差异；但该 baseline 可能包含 idle/user-wait time，且 tool/LLM 调用仍没有 start-end span，因此它只能作为下一版 preregistered packet 的 prompt-span baseline，而不是 active runtime 或完整 workflow span-duration baseline。如果这个实验失败，AgentFlame 仍可能是专家 forensic 工具，但不能写成提升普通开发者效率。

4.6 RQ5: 小模型成本和标签质量

Full run 和 R123 证明 3B 模型路径可用；R180 进一步补上本机 0.6B/1B-class/3B-class syntax-stability smoke。R122 从本机 294 个 parsed sessions 中抽取 300 个 redacted fragments：100 个 session、100 个 prompt、100 个 LLM-call。R123 用本地 llama.cpp 3B server 对这 300 个 fragments 做三次 identical repeats，共 900 次请求；结果是 900/900 valid tags，285/300 exact-stable fragments (95.000%)，模型加载约 1,002 ms，请求延迟 p50/p95 为 11/31 ms。R180 用 `--reasoning off` 对同一个 R122 fragment file 分别运行本机 0.6b、TinyLlama 1.1b 和 3b GGUF：三者都是 900/900 valid tags；exact-stable fragments 分别是 299/300、279/300 和 285/300；p95 延迟分别为 23/18/32 ms。这支持本机小模型的 syntax、latency 和 repeated-input stability smoke，但不是同一模型家族的 scaling curve。更重要的是，TinyLlama 1.1b 虽然语法有效，却大多输出 localization/localized 一类标签，说明 stability 不能替代 adequacy。R189 的 canonicalization 进一步说明，即使一个词标签语法有效，展示层仍需处理长尾碎片。该机制把 dictionary aliases、lexical+profile merges 和 review suggestions 分开报告；例如 `testcodex` 合并到 `test`，`docsupdate` 合并到 `docs`，`rootpidrefs` 合并到 `trace`。这些合并只能说明展示碎片和聚合抖动有可审计的降低 proxy，不能证明用户导航质量，也不能替代 R124 人工 adequacy labels。R190 进一步比较 raw、alias-only、lexical-only 和 profile-guarded variants，显示 lexical-only 对 LLM-event tags 过于激进 (868 个 unique tags，而当前 profile-guarded 为 1,254)；同时导出 160 行 merge-risk audit packet。R190-score 当前为 `human_labels_empty`，因此 over-merge/under-merge rate 仍需人工标注后才能报告。R196 把剩余长尾显式拆成治理动作，而不是归入 `other`：231 个 existing canonical merges、114 个 review-merge rows、39 个 regeneration candidates、2 个 contextual-split candidates、1,241 个 kept rare distinct tags 和 184 个 kept head tags。这说明长尾机制可以保留罕见但具体的信号，同时把 update、codex、ignored 这类泛化或噪声标签送入重生/拆分审核；但它仍只是治理 packet，不证明标签语义正确。R201 进一步做 policy sensitivity：在 7 个 tail/split/generic-vocabulary 变体中，baseline review-required support 为 1.926%，最大为 expanded generic vocabulary 下的 1.931%；lower/higher tail thresholds 使 long-tail support 在 0.921% 到 3.030% 间变化。这说明 review-required rows/support 在该 grid 中基本稳定，但并未测量人工审核时间；higher-tail-threshold 会把 baseline head stability 降到 65.217%，因此阈值仍是需要报告的展示策略风险。R202 用本机 llama.cpp server 对全部 41 个 regenerate/split candidates 跑候选重标，得到 41/41 grammar-valid one-word outputs、0 invalid；但这些输出仍只是候选，不更新 canonical map。R202 的顶层 summary/attempts 面向公开 artifact，嵌套的 `r196-with-regeneration/` 是 local-audit-only，因为其中可能包含 path/profile buckets。R203 进一步把这些候选变成 promotion review protocol：41 行 promotion packet、两份空 reviewer sheets、0 final labels、`long_tail_promotion_review_supported=false`，且 `canonical_map_updated=false`。R205 把这套机制转化为 reviewer 可检查的 compaction metrics：raw unique tag strings 从 1,546 降到 canonical unique tag strings 1,364，top-20 support coverage 从 93.683% 到 95.186%，long-tail

support 为 1.746%, review-required support 为 1.926%; 同时 R203 final labels 仍为 0, R190 over/under-merge rates 仍为 n/a。R209 进一步导出可逆 display-map artifact: 1,811/1,811 raw tag rows 有 active display rows, active display labels 为 1,509, 63 个 deterministic alias rows 是 active display merges, 168 个 lexical/profile merges 仍是 pending merge candidates, 41 个 regenerated labels 仍为 candidate-only, reviewed display-map diff 为 0 行, 隐藏 other rows 为 0, drilldown support preserved, 且 drilldown CSV 包含每个 display bucket 的完整 raw-tag membership。R212 对这个 session/prompt 展示策略做 ablation: raw、alias-only、profile-guarded-candidate-applied 和 R209 conservative display 四个变体都保持 183,714 total system-effect weight 不变; raw 有 26,829 个 system stacks, alias-only 和 R209 conservative display 都有 26,612 个, 假设激活所有 profile-guarded candidates 则为 26,067 个。但这个假设视图会在 2.532% 的 total effect weight 上激活未人工审核的 profile merges。因此 R212 支持的是 R209 的保守默认策略: alias 可以 active, profile merge 在人工标签回来前必须 pending; 它还没有覆盖 LLM/token display compaction。R213 进一步验证 display-mode data-layer contract: raw/display/pending 三种模式都保持 482,398 support, raw mode 有 1,811 个 buckets, display 和 pending mode 都有 1,748 个 buckets; pending mode 只叠加 209 个 candidate rows 和 323 个 review-required rows, 不改变 display membership, 且 drilldown membership 精确匹配 active display map。这组数字说明可逆 display overlay 能把长尾展示为可审计的聚合视图、提供 raw drilldown 并量化人工审核负担, 但它没有执行前端 renderer, 也不证明合并正确或标签足够好。R214 把这个长尾策略进一步变成控制环: 默认 view 保持 active-alias-only with pending overlays, 63 个 deterministic alias rows active, 168 个 profile-merge candidates 和 41 个 regenerated/split candidates pending, 323 个 rows 需要 review, 0 个 candidate merges active。它还输出一个非默认的 seven-bucket rollout preview, 将 1,811 个 raw-tag rows 和 482,398 support 按 governance state 精确分区; 同时记录 regeneration version policy, 当前 41 个 grammar-valid candidates 在没有 R203 human promotion labels 时有 0 个 promotable rows。更重要的是, R214 当前失败两个 gate: prompt-level review-required support 为 3.258%, 超过 3% 预算; higher-tail-threshold 下 head stability 只有 65.217%, 低于 80% gate。因此系统可以建议长尾合并或重生成, 但不能为了让火焰图更干净就自动提高阈值或推广 prompt-level candidates。R215 进一步检查前端 renderer-model 是否遵守同一契约: frontend/src/utils/agentflameDisplayModes.ts 在 TypeScript 下编译, 并由 Node harness 渲染 R209 display/drilldown rows, 同时 cross-check R213/R214 summary counts。该 smoke 保持 482,398 support 不变, raw/display/pending buckets 为 1,811/1,748/1,748, pending 只叠加 209 个 candidate rows 和 323 个 review-required rows, 0 个隐藏 other rows。两个负例也被拒绝: 错误 raw drilldown membership, 以及把 candidate display tag 当成 active membership。这说明 display contract 已被前端 TypeScript 模型消费, 但仍不是 browser DOM、visual click path、merge quality、adequacy 或 user utility 证据。R216 接着把同一 display contract 放入真实 headless browser: 脚本把该 TypeScript 模块编译成 browser ES

modules, 启动临时 localhost DOM harness, 用 Chrome 点击 raw/display/pending 三个模式, 并保存 DOM dump 与 screenshot。浏览器可见状态仍保持 482,398 support, pending view 显示 1,748 buckets、209 candidate overlays、323 review-required rows 和 63 active merges; 同样的 corrupted drilldown 与 candidate-promotion 负例也被拒绝。这支持的是 browser DOM harness smoke, 而不是 visual drilldown、merge quality、adequacy 或 user utility 证据。R217 进一步构建真实 Next static frontend, 提供一个由 R209 artifacts 支撑的最小 AgentFlame API fixture, 并在 headless Chrome 中打开 /agentflame。生产 AgentFlameView 默认显示 display mode, 包含 1,748 buckets、482,398 support、3 个 mode buttons, 且 display/drilldown membership 匹配。这补上 production default rendering smoke, 但仍没有点击 production controls 或验证 visual drilldown。R218 则把长尾 merge/regenerate 的 promotion 机制变成 checked gate: 它用真实 R209 pending rows 构造 synthetic review fixtures, 接受 2 个 final consensus/adjudicated preview diff rows, 拒绝 4 个 unsafe rows (unclear、weak single-label、hidden other、missing source), 保持 1,811 raw keys 和 482,398 support 不变, 并保持 canonical_map_updated=false。因为 R218 的 review labels 是 synthetic, 它只证明 reviewed display-map diff gate 的机制, 不证明 promotion quality。R219 进一步把当前证据状态机械化成 claim/RQ readiness gate: C1 为 supported, C2 为 syntax/latency supported, C3 为 mechanism supported, C4 为 fixed-suite supported, C5 为 unsupported, C6 为 partial syntax/stability only, C7 为 partial。该 gate 读取已生成 artifacts, 不读 raw traces, 不调用 LLM; 它记录 C5 responses 为 0、C6 final adequacy labels 为 0, 并保持 weak_accept_supported=false。因此 R219 的价值是把下一步固定为 P0 的 R142 participant returns 和 R124 human labels, 而不是提升论文 claim。R193/R194/R195 已把 R203 sheets 纳入 collection、preflight 和 scoring flow; 默认 run 仍没有返回标签, promotion、map update、C5 和 C6 都为 false。OSDI 版本还需要 R124 人工 adequacy labels。R184 当前把 C6 记录为 ready_for_independent_label_collection, 不是 adequacy supported; R186 将 R124 labels 排在 R142 pilot 之后或并行执行。一词标签应该被定位为 lossy navigation frames, 而不是 semantic truth。

4.7 RQ6: 开源 artifact 路径是否可用?

R160 检查的是 artifact usability, 而不是 C5 用户效用或 C6 标签正确性。我们用 8 个固定历史 Codex session 文件运行 Rust CLI, 避免当前活跃 session 在 clean/cached 两次运行之间继续增长。Clean run 写入 .agentsight/agentflame/r160-smoke-fixed, 生成 dashboard、folded stacks、SVGs 和 tags.json, 在 76 个 tag requests 中发起 60 次真实 llama.cpp calls, 耗时 1.64 s。Cached rerun 使用同一组 --session-file 输入和同一个输出目录, 76/76 tag requests 命中 cache, 0 次 LLM call, 耗时 0.11 s。artifact_usability_r160.py 进一步检查 expected files、folded total equality、redacted previews、generated report path containment、dirty raw-trace-like paths、sanitized input manifest 和 clean/cached input equality; 结果写入 docs/visexp/out/artifact-usability-r160.json。

R200 进一步把输入换成临时 public-safe Codex fix-

ture, 并由脚本管理 llama.cpp server。Clean run 发起 5 次真实 tag calls; cached rerun 为 0 次模型调用和 5/5 cache hits。该脚本显式记录没有读取真实 .codex/.claude traces, 临时 fixture 在运行后删除, committed summary 中 0 个非 redacted prompt previews, 且没有新增 raw-trace-like dirty paths。

R220 进一步把用户入口切到产品化的 agentpprof, 并从临时 clean clone 运行。脚本在 clone 内创建 public synthetic Codex fixture, 使用 deterministic regex tagger, 不调用 LLM, 也不读取真实 agent histories。它生成 tasks.pb.gz、tools.folded、tokens.json、files.folded、network.folded 和 tools.svg; 五个 projection 的样本数为 6/4/190/3/1; go tool pprof -top 能读取 tasks.pb.gz 并报告 6/6 total samples; tools/files/network/token 的 expected-stack checks、output containment 和 privacy scan 均通过。

这些结果说明 bounded local artifact path、public-safe fixture path 和本机 clean-clone agentpprof pprof readback 在固定输入下是可审计、可重复的。但它们不等于 external-machine install, 也不等于社区开发者已经能顺利使用; 本地 .agentsight report 仍包含 trace roots/session metadata, 因此不是 public-release artifact。一个更大的 36-session discovery run 首次标注耗时 173.05 s; 其 cached rerun 又出现 34 次新 LLM call, 因为当前 Codex session 在两次 discovery 之间增长。这暴露出默认体验问题: artifact smoke 和 cache benchmark 必须固定输入, 而交互式工具的默认模式应该明确采样、缓存和活跃 session 漂移。

4.8 评估完整性审计

表 4 把当前证据按 claim gate 汇总。这个表的作用不是装饰, 而是防止论文把 smoke evidence 写成系统结论。OSDI reviewer 最可能挑战的点是 C4、C5 和 C6: C4 已经有固定 command-mode suite 证据, 但还要证明 scope 不是过窄; C5 要证明用户能更快或更准确地回答 forensic questions; C6 要证明一词标签不只是语法稳定, 而是足够可用。R184/R186/R187/R188 的作用是防止过度声明: 它们把当前状态固定为 not_weak_accept, 把 launch material 与 outcome evidence 分开, 并把下一步执行顺序限定为 R142 pilot、R124 labels、R151 paper run。因此当前最诚实的定位是 “supported mechanism plus partial systems artifact”。

4.9 R183: 为什么降级本身重要

R183 的价值在于防止把 “有网络事件” 写成 “目标 workload 的网络事件已被 lineage 捕获”。R182 的低层 codex network rows 已经被 join, 但 target-specific loopback/child-process rows 仍为 0/0。因此 gate 被降级为 target-specific oracle: 只有观察到并 join 目标 loopback 或 expected child-process network rows, 才允许把 C4 扩展到 network workload coverage。R191 正是该 gate 的后续实例: 它没有把 R182 的低层 agent runtime rows 当作充分证据, 而是要求固定 Codex task 执行预置 Python HTTP probe, 并以 4/4 target rows joined、0 negative joins 作为升级条件。这体现本文的系统性: AgentFlame 不是把 trace 与 process summary 并排展示, 而是把 claim 也放入同一 lineage model 下审计, 区分用户任务 effect、agent runtime 噪声和必须保持 orphan 的事件。

5 相关工作

Flamegraphs 与 profiling. Brendan Gregg 的 flamegraph 让用户通过宽度定位 hot stacks。Grafana/Pyroscope、Datadog、Sentry 等系统已经把 flamegraph 用于 CPU、内存、profile 和 traces。AgentFlame 借用 folded stack 表示, 但 stack frame 来自 agent semantic context 与 system-effect lineage, 而不是函数调用栈。

Distributed tracing 与 agent observability. OpenTelemetry/Dapper 风格 tracing 把一次请求表示为 span tree。LangSmith、Langfuse、Phoenix、AgentOps 等工具已经记录 agent、LLM、tool、retrieval、cost 和 latency。SigNoz/Inkeep 公开展示了 multi-agent workflow 的 span-duration flamegraph。这些系统擅长解释单次 workflow 如何执行; AgentFlame 的目标是跨 session 聚合 “哪个语义任务造成哪些系统副作用”。

Token/context profiling. Context 和 token profiling 工具关注 prompt/context window 中的热点。AgentFlame 包含 token flamegraph, 但当前只作为 source-local accounting。论文主贡献是 token/semantic/tool/process/effect 共享同一 folded-stack 语法, 而不是 token cost 本身。

6 局限与下一步

当前版本仍不到 OSDI weak accept。Exact lineage 是最强系统贡献: R114 已在 20 个固定 command-mode Codex 任务上通过, R182 补上 record-mode --trace-net 的低层 agent-process network smoke, R191 又补上一个固定 Codex target-process HTTP network workload; 但 full-history exact integration、跨仓库 benchmark、更多 agent 类型和更丰富的 network workloads 仍缺。用户效用也没有 outcome data: R142 已冻结五种视图的任务包、answer key 和 scorer, R187 已生成可发放的 P01-P05 pilot launch package, R207 已确认 launch handoff 和 R195 return-file mapping, 但仍需要真实参与者响应; 纸面 C5 还需要 12-20 个参与者或明确限定的 expert study。标签 adequacy 仍需使用 blinded R124 label sheet 对 300 个 session/prompt/LLM fragments 收集人工标签; R189/R190/R190-score/R196/R201/R202/R203/R205/R209/R211/R212/R213/R214/只能说明 canonical display layer、merge-risk audit protocol、long-tail governance、policy sensitivity、候选重标链路、promotion gate、compaction metrics、可逆 display-map contract、RQ2 figure/example inputs、display-compaction ablation、display data-layer smoke、long-tail control loop 与非默认 rollup/regeneration policy、frontend renderer-model smoke、browser DOM harness、production default render smoke 和 synthetic-review update gate 已具备, 不能替代人工 adequacy、promotion quality、visual drilldown 或真实 user evidence。R160 验证本机固定输入下的 bounded local artifact path, R200 验证 public-safe generated fixture path, R220 验证本机 clean-clone agentpprof pprof readback path; 但 external-machine 安装、真实 report public sanitization、llama.cpp LLM-tag setup、默认采样策略、full write-set audit 和外部开发者反馈仍缺。

7 结论

AgentFlame 的核心不是再画一张 agent flamegraph，而是把用户语义意图、LLM/tool 历史和系统副作用放入同一个可折叠、可聚合、可审计的栈模型。当前 full run 说明该模型在真实本地 agent 历史上可运行，并且语义帧能分开 nonsemantic 和 flat baselines 大量混合的系统效应。R114 进一步说明固定 command-mode suite 中 exact semantic-effect lineage 可以做到高精度，并拒绝并发 negative controls; R182 说明同一路径在启用 record-mode `--trace-net` 后能 join 低层 agent-process network rows; R191 说明固定 Codex target-process HTTP network rows 也能被 join。R160、R200 和 R220 说明 bounded artifact path、public-safe fixture path、cache rerun 和本机 clean-clone pprof read-back 已经可审计，但还不是社区级 artifact 结论。但顶会版本必须继续证明 broader/full-history exact lineage、标签 adequacy 和用户任务收益。在当前证据下，最诚实的论文定位是：一个有完整系统化方向和真实 characterization 的语义系统效应 profiler，而不是已经完成的 OSDI artifact。

参考文献

- [1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- [2] Benjamin H. Sigelman et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report, 2010.
- [3] SigNoz. Understanding Flame Graphs for Visualizing Distributed Tracing. <https://signoz.io/blog/flamegraphs/>.
- [4] SigNoz. How Inkeep Monitors Their AI Agent Framework with SigNoz. <https://signoz.io/blog/inkeep-ai-agent-monitoring/>.
- [5] Datadog. Trace View. https://docs.datadoghq.com/tracing/trace_explorer/trace_view/.
- [6] LangChain. LangSmith tracing documentation. <https://docs.langchain.com/langsmith/>.
- [7] Langfuse. Observability documentation. <https://langfuse.com/docs/observability>.
- [8] Arize Phoenix. Tracing documentation. <https://arize.com/docs/phoenix>.

表 4: 当前 claim gate。Supported 表示已有当前 artifact 支持; partial 表示机制成立但 scope 仍窄; unsupported 表示缺少必要实验。

Claim	当前证据	Verdict	缺口
C1 folded aggregation	167,005 system observations 折叠为 24,295 semantic stacks; sampled artifact verifier 通过。	supported	需要把 full-run artifact packaging 固化成 fresh-clone release workflow。
C2 one-word tags	2,463 prompt rows 和 90,930 LLM calls 全部满足一词 grammar; 0 final tag failures。	supported	Syntax 不等于 adequacy; 需要人工标签质量评估。
C3 semantic partitioning and display contract	Nonsemantic/flat baselines 分别有 90.219% 和 90.770% mixed weight; R224/R170 中 prompt-only 将 system bucket/residual mixed weight 降到 36.722%/7.485%; R211 给出 RQ2 figure inputs, 其中 <code>git/read/ok</code> 有 116 个 prompt tags, <code>cargo/test/ok</code> 有 48 个 prompt tags; R209/R212 证明可逆 display map 与保守 display policy: R209/alias-only 为 26,612 stacks, profile-guarded hypothetical 为 26,067 stacks 但会激活 2.532% 未审查 effect weight; R223 将这些结果汇总为 projection tradeoff; R225 生成 prompt-span duration baseline, top-10 duration/effect tags 只重合 7/10, Spearman 为 0.623。	partitioning supported; display mechanics supported	需要更多仓库和用户任务证明不是单仓库 workload 特例; R213-R218 只是 renderer/data contract 和 update-gate smokes, 不是 C3 scientific outcome evidence; 还需要 LLM/token display-compaction ablation、真实用户交互证据, 以及 R190/R203 标签返回后的 merge/promotion quality。
C4 exact lineage	R114 在 20 个固定 <code>codex</code> tasks 上 join 1,273/1,273 in-scope effects, 0 false positives, 0 false negatives, 0/3,170 negative-control effects joined; R182 join 35/35 低层 <code>codex</code> process network rows, 0 network orphans, 0/604 negative-control effects joined; R191 在固定 Codex HTTP probe 上 join 4/4 target <code>python3</code> network rows, 0/310 negative-control effects joined。	fixed-suite supported	需要 full-history integration、跨仓库 replication、更多 agent 类型和更丰富的 target-specific network workloads; R191 不证明 HTTP payload/URL reconstruction 或任意 network workload。
C5 user utility	已有 task packet、scorer、preregistration、R187 P01-P05 launch package 和 R207 return-file handoff audit; 尚无 participant responses。	unsupported	需要基于已冻结的 preregistration 运行 trace tree/event-count/flat/nonsemantic/semantic 视图的 user benchmark。
C6 tag adequacy	R180 在本机 0.6b/1.1b/3b GGUF 上得到 2,700/2,700 valid tags; exact-stable fragments 为 299/300、279/300、285/300, p95 为 23/18/32 ms; 1.1b 存在 localization-like collapse; R189/R196/R201/R202/R203/R205/R209/R212/R218 只支持 display-layer noise-control/governance/sensitivity/regeneration-smoke/promotion-protocol/compaction metrics/display-map contract/compaction ablation/update-gate mechanics。	partial	需要 R124 人工 adequacy labels; 若要声称 controlled model scaling 还需同族模型 benchmark。
C7 artifact usability	R160 bounded smoke 生成 expected artifacts; clean run 1.64 s, 60 uncached LLM calls; cached rerun 0.11 s, 76/76 cache hits, 0 LLM calls。R200 public-safe fixture smoke clean run 5 次 tag calls, cached rerun 0 次模型调用和 5/5 cache hits。R220 本机 clean-clone <code>agentpprof</code> smoke 生成 pprof/folded/JSON/SVG outputs, tasks/tools/tokens/files/network samples 为 6/4/190/3/1, go tool pprof readback 和 expected-stack checks 通过。	partial	仍需要 external-machine clean-install workflow、real-report public sanitization、llama.cpp LLM-tag setup、稳定默认采样、full write-set audit 和外部开发者反馈。